

Introducción al Plan de Ejecución de Imrryr OS

Basado en nuestro documento maestro, el Proyecto Imrryr OS.pdf, nos enfrentamos a la construcción de un ecosistema local-first diseñado para ser el núcleo cognitivo y operativo de una empresa. En lugar de crear un sistema desde cero, la estrategia maestra radica en utilizar el proyecto de código abierto OpenCode como infraestructura base (nuestro "Arnés" central). Esto nos otorga compatibilidad nativa con el estándar MCP (Model Context Protocol) y neutralidad de modelos, permitiéndonos alternar entre APIs como Gemini Pro o modelos locales como Qwen según la necesidad.

A continuación, presento el plan de realización completo y extremadamente atomizado para Imrryr OS.

FASE 1: Preparación del Motor Base (Hackeo de OpenCode)

El objetivo aquí es tomar la arquitectura de OpenCode, aislarla en tu disco duro para garantizar la soberanía de datos y prepararla para ser un sistema "invisible" en el backend.

- 1.1. Clonación y Entorno Seguro
 - Realizar un fork del repositorio original de OpenCode para tener una copia de desarrollo propia.
 - Crear un entorno virtual (Docker ligero o venv en Python) exclusivo para Imrryr OS.
 - Configurar el archivo `.env` en el directorio raíz, donde vivirán las credenciales locales de forma aislada.
- 1.2. Configuración del Traductor Agnóstico
 - Instalar e integrar la librería LiteLLM dentro de la capa de enrutamiento de OpenCode.
 - Configurar la conexión a través de LiteLLM para modelos de nube (ej. Gemini).
 - Configurar la conexión a través de LiteLLM para modelos locales corriendo en la GPU (ej. Ollama/Qwen).
- 1.3. Encapsulamiento del Backend
 - Crear un script de inicio (`startup.py` o similar) que ejecute OpenCode en segundo plano (modo headless o backend invisible).
 - Desactivar temporalmente la interfaz nativa de terminal de comandos de OpenCode para evitar conflictos con el futuro dashboard visual.

FASE 2: Arquitectura de la Bóveda de Conocimiento

Aquí crearemos las estructuras donde residirá la memoria, evitando la dependencia de servidores de terceros y el colapso cognitivo (Context Bloat).

- 2.1. Estructura de Directorios Raíz

- Crear la carpeta local `/agentes`.
- Crear la carpeta local `/skills`.
- Crear la carpeta local `/semillas`.
- 2.2. Implementación de Memoria Vectorial (RAG)
- Instalar y configurar ChromaDB para operar 100% local.
- Crear un script de ingesta de datos que tome PDFs y manuales de museografía y los convierta a coordenadas matemáticas dentro de ChromaDB.
- 2.3. Implementación de Memoria Relacional
- Generar un archivo de base de datos SQLite en el directorio local.
- Crear la estructura de tablas para "Gastos", incluyendo columnas precisas para: fecha, monto, comercio y categoría.
- 2.4. Motor de Autodocumentación de Proyectos
- Crear un formato base en Markdown (`.md`) para los Archivos de Estado de los proyectos de software.
- Programar un script automatizado que, al final de cada sesión, lea el trabajo realizado y sobrescriba el archivo `.md` resumiendo tecnologías usadas, decisiones lógicas y variables creadas.

FASE 3: Desarrollo de la Biblioteca de Skills (MCP)

Desarrollaremos los scripts de Python atómicos (de 20 a 50 líneas) que funcionarán como las "manos" del sistema.

- 3.1. Skills Financieros e Investigación
- Programar `leer_inbox_gmail()` usando la API de Gmail para extraer el texto de los correos bancarios.
- Programar la función de inyección de datos que tome los montos extraídos y los inserte en SQLite.
- Programar un script de web scraping con BeautifulSoup o Playwright que navegue por los portales de Sercotec y Corfo leyendo bases de fondos.
- 3.2. Skills de Comunicación y Presencia
- Programar `convertir_texto_a_audio()` integrando un modelo local de Text-to-Speech (TTS).

- Crear un skill que reciba un texto formateado y lo envíe vía API al CMS local (WordPress) para crear entradas de blog.
- Crear un skill de publicación para enviar resúmenes a las APIs de LinkedIn o X.
- 3.3. Bus de Eventos y Alarmas
- Programar `crear_evento_calendario()` para que genere y guarde archivos `.ics` en el disco local.
- Levantar un Webhook (una dirección web invisible) diseñado para recibir "golpes" de información en tiempo real cuando llega un comprobante del banco.
- 3.4. Creación de Manifiestos MCP
- Redactar un archivo de texto en formato JSON para cada skill anterior, sirviendo como "manual de instrucciones" para que el Enrutador sepa qué parámetros enviar a cada herramienta.

FASE 4: Hackeo y Especialización de Micro-Agentes

Reconfiguraremos la jerarquía nativa de OpenCode para implementar el sistema de Carpetas Dinámicas (Hot-Plugging).

- 4.1. Configuración de Carpetas Dinámicas
- Modificar el enrutador central de OpenCode para que, al encenderse, realice un escaneo en la carpeta `/agentes` buscando archivos `.yaml`.
- Diseñar la estructura del archivo manifiesto `.yaml` que incluya: `nombre`, `descripcion` y el array de `herramientas_permitidas`.
- 4.2. Especialización del Escuadrón
- CTO Adjunto: Mantener al agente primario `Build` de OpenCode con permisos elevados de escritura para codificación y depuración en Docker ligero.
- Agente Financiero: Crear `agente_financiero.yaml`, dándole acceso exclusivo al skill de lectura de Gmail y escritura en SQLite.
- Agente Investigador: Crear su manifiesto, asociándolo al skill de web scraping y programando un cron job para ejecución diaria.
- Agente Creativo: Crear su manifiesto instruyéndolo para recibir ideas sueltas, etiquetarlas (ej. `[hardware]`, `[videojuegos]`) y guardarlas en la carpeta `/semillas`.
- Agente Comercial (CRM): Crear su manifiesto para que acceda a la base de datos vectorial (proyectos pasados) y genere propuestas/cotizaciones en formato PDF.

- Guardia de Seguridad: Configurar un micro-agente silencioso encargado de la observabilidad. Programarlo para detener cualquier proceso de scraping que supere los 5 minutos y evitar el gasto descontrolado de tokens.

FASE 5: Pasarela Móvil (WhatsApp Gateway)

Construiremos el puente asincrónico que conecta el hardware local con el teléfono móvil.

- 5.1. Conexión Base de WhatsApp
 - Instalar la herramienta `whatsapp-web.js` (Node.js) o su librería equivalente en Python dentro del directorio `/gateway`.
 - Programar el servicio que genere y exponga el código QR para escanear y vincular tu número de teléfono.
- 5.2. Recepción y Transcripción de Audio
 - Crear un listener (escuchador) que intercepte mensajes de voz entrantes en formato `.ogg`.
 - Programar un script que pase el archivo `.ogg` por el modelo de transcripción local Whisper para pasarlo a texto.
 - Enrutar ese texto transcrito directamente hacia el Agente Creativo para su guardado en `/semillas`.
- 5.3. Interacción y Recordatorios Proactivos
 - Programar la lógica para que, si el usuario escribe "Léemelo en audio", el sistema ejecute el skill de Text-to-Speech y envíe un archivo de voz de vuelta al chat.
 - Crear una función que analice instrucciones de tiempo (ej. "Recuérdame revisar el presupuesto a las 5 PM") y deje un recordatorio programado que dispare un mensaje estructurado hacia WhatsApp a la hora señalada.

FASE 6: El Wrapper (Dashboard de Escritorio)

Construiremos la interfaz gráfica modular y personalizable.

- 6.1. Servidor Local y Conectividad
 - Levantar un servidor web local utilizando FastAPI (o Node.js) que se comunique con el motor oculto de OpenCode.
- 6.2. Maquetación Visual y Componentes
 - Escribir el código HTML y JS utilizando librerías de componentes arrastrables (Drag & Drop) para que los bloques puedan moverse como piezas de un tablero.

- Aplicar reglas de CSS que consoliden una estética oscura, melancólica, minimalista y lofi
- 6.3. Integración de Widgets Dinámicos
- Programar un "Widget Financiero" que consulte la base SQLite y dibuje el gráfico de gastos.
- Programar un "Widget de Semillas" que muestre la lista interactiva de la carpeta `/semillas`.
- Programar un "Widget de Terminal" que funcione como chat directo con el Agente Codificador `Build`.
- Vincular el Front con las Carpetas Dinámicas: programar la lógica para que si un archivo `.yaml` es borrado del directorio local, su widget correspondiente desaparezca automáticamente del dashboard.

FASE 7: Marca Blanca y Empaquetado (White-Labeling)

El paso final para que el AI-OS pueda ser distribuido y comercializado a terceros.

- 7.1. Limpieza y Bloqueo de Perfiles
- Desarrollar un script que empaquete una versión "Perfil Pyme/Comercial".
- Configurar este script para que, al ejecutarse, "borre" o desactive automáticamente al agente `Build` y las herramientas de terminal para evitar manipulaciones de código.
- 7.2. Generación del Ejecutable Independiente
- Asegurar que el instalador copie solo las carpetas necesarias (Agente Financiero, CRM, Bus de Eventos, WhatsApp).
- Configurar el empaquetado para que cada instalación genere bases de datos SQLite y ChromaDB vacías y aisladas, garantizando un nodo autónomo sin cruce de datos.

Este es el plan de construcción absoluto de Imrryr OS, dividido en las tareas más pequeñas posibles.